

C++ GAME SERVER · PORTFOLIO

게임 서버 개발 포트폴리오

대표 · NETWORK

IOCP 네트워크 라이브러리

IOCP 네트워크·게임 라이브러리 구현과 여러 서버 장시간 안정성 검증

대표 · MMORPG

MMORPG 월드 서버

위 라이브러리를 실제 월드 서버에 적용, 최대 4000명 부하로 병목 측정·개선

기반 · CONCURRENCY

Lock-Free 자료구조

동시성 문제 4종 재현·분석·해결

기반 · OPTIMIZATION

A* 최적화

측정 공정성 확보 + 단계별 최적화 약 30%

프로젝트 구성 요약

각 프로젝트가 무엇을 구현하고, 무엇을 측정했으며, 무엇을 개선했는지 한눈에 정리

Lock-Free 자료구조

역할 게임 서버의 멀티스레드 기반
구현 락프리 Stack·Queue·메모리 풀
결과 동시성 문제 4종 재현·해결

A* 최적화

역할 성능을 측정으로 다루는 능력
구현 공정한 측정 환경 + 단계별 최적화
결과 평균 처리시간 약 30% 감소

IOCP 네트워크 라이브러리

역할 재사용 가능한 서버 프레임워크
구현 IOCP 네트워크·게임 라이브러리 + 검증 서버군
결과 장시간 무중단 최대 7일 검증

MMORPG 월드 서버

역할 라이브러리의 실제 적용 결과물
구현 AOI·서버 권위 전투·비동기 DB 저장
결과 최대 4000명 부하로 병목 측정·개선

Lock-Free 개요

락프리 자료구조에서 실제로 발생하는 동시성 문제를 재현하고 분석

ABA 문제

같은 주소가 재사용될 때
CAS가 잘못 성공하는 문제

Page Decommit

참조 중인 메모리가
OS에 반환되어 크래시

Order Reversal

삽입 진행 중 노드를
오인해 순서가 꼬임

Shared Pool 오염

공유 노드 풀에서
소유가 뒤섞이는 문제

이 프로젝트의 초점

- 락프리 Stack·Queue·메모리 풀을 직접 구현
- 핵심은 구현이 아니라 **문제 재현·원인 추적**
- 각 문제를 재현 코드 ↔ 해결 코드 쌍으로 정리
- 생성한 락프리 자료구조를 실제 타 포트폴리오 프로젝트에 적용

Stack / Queue 구조

CAS로 락 없이 처리하되, 태그로 재사용을 구분하고 자체 풀로 노드를 재사용

Lock-Free Stack (Push / Pop)



Lock-Free Queue (Enqueue / Dequeue)



스택은 **top**을 **CAS**로 교체하며 Push/Pop을 처리합니다. 포인터 상위 비트에 단조 증가하는 **태그(Tag)**를 실어, 주소가 같아도 Tag가 다르면 CAS가 실패하게 만듭니다.

큐는 **Head**를 **CAS**로 교체하여 Dequeue를 처리합니다. 또한 Enqueue 작업 시 Tail의 Next에 newNode를 연결하기 위해 **1st CAS**를 수행하고, Tail 변수를 원자적으로 변경하기 위해 **2nd CAS**를 진행합니다.

그림은 **포인터 = 실제 주소 + 태그**로 읽으면 됩니다.

ABA 문제

같은 주소가 재사용될 때 CAS가 잘못 성공하는 문제를, Tag로 구분해 차단

문제 발생 시나리오 (스택)

① 스레드 A가 Pop 준비 — top=N1, next=N2 스냅샷



② 그 사이 스레드 B가 N1·N2를 Pop(풀 반납)하고 N1을 재활용해 다시 Push



③ A의 CAS(top: N1→N2) — 주소가 같아 잘못 성공



스택 파손 · 이중 반납 → 크래시

④ 해결 — 포인터에 태그 붙이기: N1(v1) ≠ N1'(v2)이므로 CAS 실패·재시도



스택 Pop이 top과 그 next를 읽은 사이, 다른 스레드가 그 노드를 빼고 반환했다가 다시 넣으면 top이 A → B → A로 돌아옵니다. 주소만 비교하는 CAS는 그 변화를 못 잡아 성공해버립니다.

원인은 크래시 상태만으로는 알 수 없어, 각 스레드가 **바라본 Top이 무엇이고 Top을 무엇으로 바꾸었는지** 랙프리 스택에 로그 메모리 배열을 만들어 원자적으로 기록하여 확인했습니다. 해결은 포인터에 태그를 붙여 ‘주소 + 태그’를 함께 비교하는 것입니다.

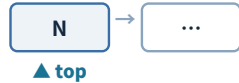
재현 장치: 재현용 메모리 풀을 **큐 방식**으로 만들어, Free 시 반납하려는 노드가 풀의 맨 앞(front)에 이미 있으면 false를 반환하게 했습니다 — ABA가 만든 **이중 반납**을 감지합니다.

Page Decommit 문제

참조 중인 메모리가 OS에 반환되는 문제는, Tag 카운터가 아니라 자체 풀로 차단

문제 발생 시나리오 (스택 + OS 페이지)

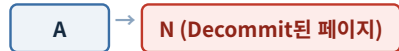
① 스레드 A가 Pop 준비 — top=N 스냅샷



② 스레드 B가 N을 Pop해 OS에 반납 → OS가 추후 해당 페이지를 Decommit



③ A가 저장해 둔 Top 주소로 N→next를 참조 — 이미 사라진 메모리



액세스 위반 크래시 — 재사용이 아니라 소멸이라 Tag 카운터로는 못 막음

④ 해결 — 노드를 OS에 돌려주지 않는 자체 풀 보존 → 읽은 주소가 항상 유효



노드 페이지를 직접 관리하는 스택에서, 마지막 노드 반환 시 페이지를 OS에 반환하면 문제가 생깁니다. 스레드 A가 top을 읽은 뒤, 스레드 B가 해당 주소가 가리키는 노드가 있는 메모리를 OS에 반납하고 OS가 추후 해당 페이지를 Decommit 시키면, A 스레드가 스택에 저장한 Top 주소값을 가지고 next 변수를 참조할 때 크래시가 발생합니다.

ABA는 ‘같은 주소 재사용’이라 Tag로 막지만, 이런 ‘참조 중 메모리가 사라지는’ 문제라 Tag 카운터로는 못 막습니다. 그래서 노드를 OS로 돌려주지 않는 **자체 메모리 풀**로 회수하여, 참조 시 해당 메모리가 Decommit 되지 않게 했습니다.

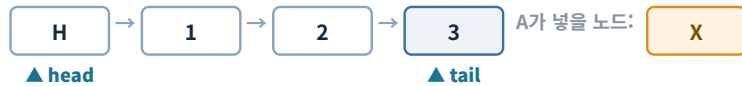
핵심 구분: **메모리 소멸 ≠ 재사용**. 방어 수단도 서로 다릅니다. 재현 장치: VirtualAlloc으로 **페이지 1개(4KB)**를 직접 할당해 노드 배열로 쓰고, 노드가 전부 반납되는 순간 재현 코드가 직접 **VirtualFree(MEM_DECOMMIT)**를 호출 → 이후 next 참조에서 액세스 위반이 나게 구성했습니다.

Order Reversal 문제

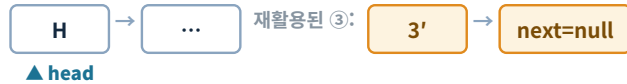
삽입 진행 중 노드를 다른 스레드가 완성된 꼬리로 오인해 순서가 꼬이는 문제를, next를 0xFFFF...로 미는 방식으로 막고 태그 방식으로 개선

문제 발생 시나리오 (큐)

① 스레드 A가 Enq(X) 준비 — 그 순간의 tail=③, ③.next==nullptr 스냅샷



② 다른 스레드가 Deq로 ③까지 빼서 풀에 반납 → ③이 재활용되어 새 Enq에 쓰이며 next를 nullptr로 미는 순간



③ A의 1st CAS(③.next: nullptr→X)가 그냥 성공 — X가 실제 큐가 아닌 재활용 노드에 붙음



A는 성공했다고 믿지만 X는 실제 큐에 없음

④ 이후 Y가 정상 Enq — 꺼내 보면 나중에 넣은 Y가 X보다 먼저 나옴



락프리 큐 Enqueue는 Tail 노드에 신규 노드 연결 → Tail 전진 두 단계로 나뉩니다. 스레드가 Tail 노드의 next 변수를 1st CAS 직전에 스택에 저장했는데, 해당 노드가 다른 스레드에 의해 재활용되고 Enqueue 시 재활용되어 next == nullptr로 초기화되는 순간 기존 스레드의 1st CAS가 성공하게 되고, 큐가 아닌 재활용된 노드에 본인이 할당받은 노드를 삽입하여 순서가 뒤바뀌게 됩니다.

원인은 크래시 상태만으로는 알 수 없어, 각 스레드가 Enqueue/Dequeue를 시도할 때 락프리 큐 멤버에 로그 메모리 배열을 만들어 그 당시 바라본 Head, Tail, TailNext가 무엇이고 Head, Tail을 무엇으로 바꾸었는지 기록하여 확인했습니다. 해결은 할당 받은 노드의 _next를 0xFFFF...(삽입 진행중 표식 값)으로 초기화하는 것입니다. 1st CAS가 성공한 뒤에야 _next를 nullptr로 확정해 다른 스레드가 1st CAS 가능하게 합니다.

개선(태그 방식): 문제의 본질이 'nullptr의 세대 구분'이므로, _next를 nullptr로 만들 때 상위 17비트에 카운터(태그)를 실어 기록하고 1st CAS가 '내가 본 태그 붙은 nullptr'인지 체크하게 개선 — 재활용 노드의 새 nullptr는 태그가 달라, 순서 뒤바뀔의 원인이 되는 CAS가 실패합니다. Tag는 next 변수가 수정됨을 인지하기 위한 것이라, 1st CAS할 때 newNode에까지 Tag를 붙일 필요는 없습니다.

Shared Node Pool 오염

여러 큐가 노드 풀을 공유할 때의 소유 혼동을, Qid+128비트 CAS로 막고 전역 태그 발급 방식으로 개선

문제 발생 시나리오 (두 큐 + 공용 풀)

① 큐1의 스레드 A가 Enq(P) 준비 — tail=N, N.next==nullptr 스냅샷



② 다른 스레드가 큐1에서 N을 Deq → 공용 노드 풀에 반납



③ 큐2가 N을 할당받아 자기 큐에 Enq — N의 next를 nullptr로 미는 순간



④ A의 1st CAS(N.next: nullptr→P)가 성공 — 큐1에 넣으려던 P가 큐2에 붙음



다른 큐에 노드를 삽입하는 오염 — next==nullptr만으로는 재활용된 것을 모름

태그 발급 카운터를 큐마다 두면 생기는 문제

① 큐1의 A가 스냅샷 — N.next = null | tag5 (큐1의 발급 카운터가 5 발급)



② N이 풀을 거쳐 큐2로 재활용 — 큐2의 발급 카운터도 우연히 5 → N.next = null | tag5



③ A의 1st CAS 비교값(null|tag5)이 우연히 일치 → 태그를 달았는데도 오인 삽입

해결: 발급 카운터를 static 공용 변수로 → 태그가 큐를 가로질러 유일

큐별 발급이면 서로 다른 큐가 같은 태그를 만들 수 있음

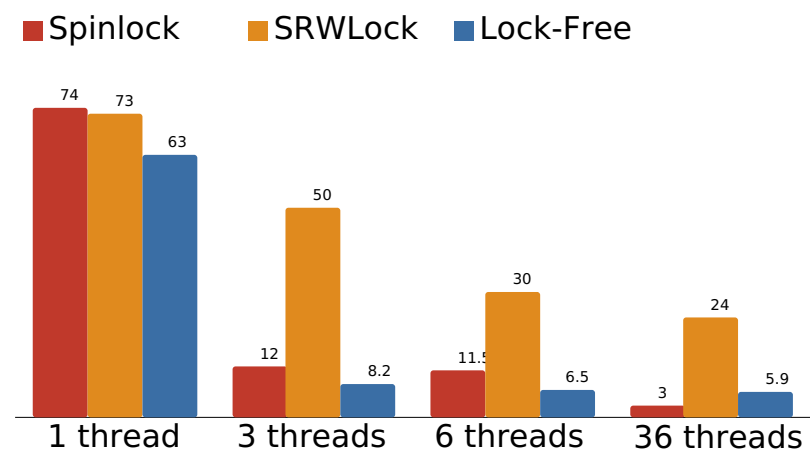
큐 노드 풀이 공유되면, 한 큐가 반환한 노드를 다른 큐가 재사용합니다. 이때 원래 큐의 스레드가 1st CAS를 하기 위해 저장했던 next 변수가 속한 노드가 실제로는 다른 스레드에 재활용된 노드인데, 당시 자기가 본 노드의 next 변수로 착각하고 연결하면 **다른 큐에 노드를 삽입하는 오염**이 생깁니다.

해결은 노드에 **소유 큐 식별자(Qid)**를 심고, 1st CAS를 **128비트 CAS**로 바꿔 next와 Qid를 함께 검사하는 것입니다. 남의 큐 노드면 CAS가 실패해 거부됩니다.

개선(태그 + 전역 발급): next의 nullptr에 태그를 심는 방식을 공용 풀에 적용하되, 태그 발급 카운터를 큐별로 두면 왼쪽 아래 시나리오처럼 서로 다른 큐가 같은 태그를 발급해 오인 삽입이 가능하므로, **static 공용 발급 카운터**로 모든 큐가 하나의 발급 변수를 공유하게 했습니다. 태그가 큐를 가로질러 유일해져 Qid 없이 **CAS64**만으로 차단됩니다.

동기화 방식 비교 — 결과

스레드 수를 늘려 가며 측정하면, 세 방식의 처리량은 서로 다른 곡선



3분 Total Count(단위: 억) · 논리 12코어 · 스레드 6개 이하는 물리 코어 고정 · 계속 카운터 캐시라인 분리 후 재측정

방식	스레드 1 → 3 → 6 → 36개 변화
Spinlock	74억 → 12억 → 11.5억 → 3.0억 — 경합 시작과 함께 급락, 6개까지 정체하다 코어 초과에서 붕괴
SRWLock	73억 → 50억 → 30억 → 24억 — 전 구간 1위, 완만하게 하락
Lock-Free	63억 → 8.2억 → 6.5억 → 5.9억 — 급락 후 완만, 코어 초과(36개)에서 Spinlock을 역전

같은 push+pop 작업을 세 방식으로 3분간 반복한 처리량입니다. 왜 이런 차이가 나는지는 다음 네 페이지에서 단 일 → 3개 → 6개 → 36개 순으로 분석합니다.

비교 분석 ① 단일 스레드

경합이 없어도 차이가 나는 이유는 명령 종류가 아니라 루프당 원자 연산 개수

74억

Spinlock

잠금 1회 + 해제 1회

73억

SRWLock

acquire + release

63억

Lock-Free

topCnt 2회 + CAS 2회

경합이 없는데도 Lock-Free만 13%쯤 낮습니다. 처음에는 lock 명령어(xchg)와 CAS의 **명령 자체 부하 차이**를 의심해, 별도 코드로 3분 동안 한쪽은 InterlockedExchange 2회, 한쪽은 CAS 2회만 반복시켜 비교했습니다 — 결과는 **차이가 거의 없음** (양쪽 모두 약 157억 회).

그러면 원인은 명령 종류가 아니라 **개수**입니다. 락 방식은 루프 1회에 원자 연산이 2회(잠금·해제)인데, 락프리 스택은 topCnt 할당에 interlock 2회 + CAS 2회로 **4회**가 들어갑니다.

원자 연산이 개당 비용을 갖는 이유: 일반 store는 스토어 버퍼에 기록만 하고 넘어가지만, lock이 붙으면 **캐시까지 반영하고 순서를 보장**해야 해서 그만큼 시간이 더 듭니다. 단일 스레드에서 interlock 몇 개 차이가 13%를 만든 이유입니다.

확인한 것

- xchg 2회 vs CAS 2회 반복 → 차이 거의 없음(명령 자체 부하 동일)
- 락 2회 vs 락프리 4회 — 원자 연산 개수 차이
- lock 연산은 스토어 버퍼 기록으로 안 끝나고 캐시 반영·순서 보장 비용을 냄

비교 분석 ② 3스레드

경합의 실체는 캐시라인 독점 대기 — 백오프 이식 실험으로 원인 검증

locked 연산은 대상 캐시라인을 **독점(M 상태)**해야 실행됩니다. 여러 코어의 locked 연산이 같은 라인에 겹치면, 한 코어가 라인을 쥐고 처리하는 동안 **다른 코어에 붙은 스레드는 러닝 상태로 대기**합니다.

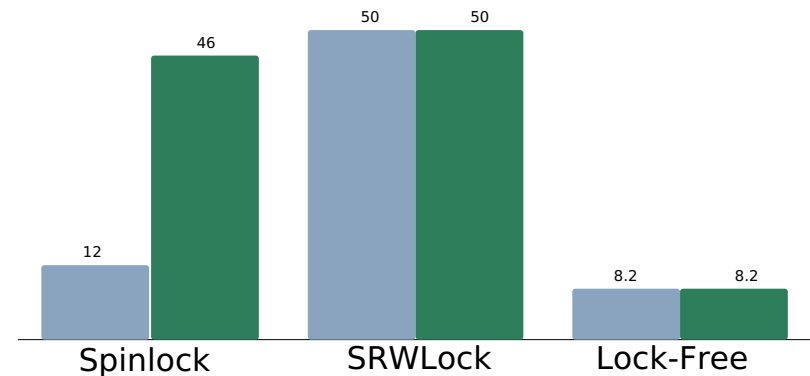
Spinlock(12억, 성공률 ~11%): 실패하면 즉시 재시도하는데, 실패한 **InterlockedExchange**조차 라인을 독점으로 가져오며 다른 코어의 캐시라인을 무효화합니다. 그래서 ① 락 소유 스레드가 unlock하려면 무효화된 라인을 다른 캐시에서 다시 가져오는 딜레이가 생기고, ② 실패한 스레드끼리도 의미 없이 서로의 라인을 계속 무효화해 실패한 스레드의 xchg도, 소유 스레드의 해제도 함께 느려집니다. 시도의 9할이 이런 폭풍이라 임계구역이 체감상 길어집니다.

SRWLock(50억): 디스어셈블리로 확인하면 fast path는 lock bts 1회, 경합 시 스레드를 바로 block하지 않고 **RtlBackoff**(자기 스택만 읽는 pause 지연 루프, 호출마다 지연 지수 증가)를 거쳐 재시도 — 소유권 획득 시도를 스레드마다 시간축으로 분산시켜, 여러 스레드가 동시에 lock 명령 처리에 들어가 대기하는 상황을 최대한 막습니다.

Lock-Free(8.2억, 성공률 70%대): 실패가 적은데도 느린 이유는 실패 재시도가 아니라 **성공하는 연산 자체의 비용**입니다. ① push/pop마다 topCnt 할당에 interlock이 2회 더 들어가는데, 이 연산은 스토어 버퍼 플러시 + 캐시라인 독점을 요구하고, 내 캐시라인에 기록하면서 다른 코어의 사본을 invalid시켜 **다른 스레드가 topCnt를 할당할 때 라인을 다시 가져오는 딜레이**를 만듭니다. ② 성공률이 높다는 건 성공한 CAS가 계속 내 캐시라인을 수정하며 다른 캐시를 invalid로 만든다는 뜻이라, 그 코어가 다음 CAS를 처리할 때 캐시라인을 매번 다시 가져와야 하는 부하가 반복됩니다.

검증으로 Spinlock에 SRWLock과 같은 지수 백오프를 이식하자 **12억 → 46억**(성공률 11% → 81%)으로 SRWLock에 근접했습니다. 같은 백오프를 Lock-Free에 넣으면 **8억대 그대로** — 두 방식의 병목이 다르다는 것이 분리 검증됩니다.

■ 백오프 전 ■ 백오프 후



3스레드 · 지수 백오프(pause, 실패 시 2배) 이식 전/후 Total Count(억)

비교 분석 ③ 6스레드

스핀락은 이미 포화라 그대로, 락프리는 경쟁 스레드 증가만큼, SRWLock은 완만하게 하락

11.5억

Spinlock (3개와 동일)

성공률 10~12%도 그대로

30억

SRWLock

독주 구간이 짧아져 완만 하락

6.5억

Lock-Free

성공률 70% → 50%대

Spinlock이 변하지 않는 이유: 같은 캐시라인에 대한 locked 연산은 특정 시점에 1코어만 수행할 수 있으므로, **초당 시도 횟수에 상한**이 있습니다. 3스레드 시점에 이미 그 상한에 포화됐기 때문에(성공률 ~11%), 스레드를 6개로 늘려도 같은 파일을 6명이 나눠 가질 뿐 총량·성공률이 그대로입니다.

Lock-Free의 성공률이 50%대로 떨어진 이유: 내가 스냅샷을 찍고 CAS하는데, **뒤이어 스냅샷을 찍으려 들어오는 스레드 개수가 늘어나기 때문**입니다. 이렇게 들어온 스레드들은 보통 CAS에 실패하니 실패율이 늘어납니다.

SRWLock(30억): 스레드 개수가 늘어남에 따라 경쟁 빈도가 높아져 카운트 합은 하락하지만, **backoff 기능으로 Spinlock처럼 크게 하락하는 상황을 방지**합니다.

핵심

- 같은 캐시라인 locked 연산의 초당 상한 → 3개 시점에 이미 포화
- 락프리 실패율 상승 = 함께 스냅샷을 찍으려 들어오는 경쟁 스레드 증가
- SRWLock = 경쟁 빈도만큼 하락하되 backoff로 급락 방지

비교 분석 ④ 36스레드 (코어 초과)

락 보유 스레드가 선점당하는 순간부터, 진행을 보장하는 구조가 승리

3.0억

Spinlock · 성공률 2%

락 보유 스레드 선점 → 봉괴

5.9억

Lock-Free · 성공률 80%대

진행 보장으로 Spinlock 역전

24억

SRWLock

block 양보 + backoff로 방어

스레드 36개는 논리 코어(12개)의 3배라 스케줄러가 개입하는 구간입니다. 컨텍스트 스위치가 초당 수십 회에서 1,400~3,700회로 됩니다.

Spinlock 봉괴(3.0억): 락을 쥔 스레드가 임계구역 도중 선점되면 락이 잠긴 채 방치되고, 나머지 스레드들이 쿼텀을 다 태우며 헛돌아 **보유 스레드의 복귀에 필요한 CPU 시간까지 잡아먹습니다.**

Lock-Free 역전(5.9억): 어떤 스레드가 선점돼도 남은 스레드는 진행 가능하고, 노드 할당도 각자 독립적으로 합니다. 성공률이 오히려 80%대로 오르는 이유는 두 가지입니다. ① 6스레드에서는 각 스레드가 코어를 독점해 쿼텀을 다 써도 대신 돌릴 스레드가 없어 곧바로 다시 쿼텀을 받아 이어 달렸고, 그래서 한 스레드가 CAS하는 사이에 **스냅샷을 찍으려 들어오는 스레드가 잦아** 실패율이 올랐습니다. 36스레드에서는 쿼텀이 끝나면 컨텍스트 스위칭으로 도는 스레드가 바뀌기 때문에, **끼어들려던 스레드가 스냅샷을 찍기 전에 밀려나고 CAS하던 스레드는 흐름을 그대로 이어가는** 경우가 늘어 성공률이 올라갑니다. ② 6스레드에서는 스레드마다 L1 캐시가 분리되어, 한 스레드가 CAS에 성공하면 다른 코어의 캐시라인이 무효화되고 스냅샷 때마다 라인을 다시 가져와야 해 처리 시간이 늘어집니다. 36스레드에서는 물리 코어 하나에 스레드 2개가 붙어 **L1을 공유**하므로, 같은 코어의 짝 스레드는 CAS 결과를 다른 코어에서 가져올 필요 없이 바로 읽어 **스냅샷→CAS 처리 시간이 단축**됩니다.

SRWLock(24억): 경합에 실패한 스레드가 스펙처럼 쿼텀을 태우며 소유 스레드를 방해하는 대신 그대로 block되어 코어를 양보합니다. 여기에 backoff까지 더해져 스펙처럼 크게 하락하지 않습니다.

이 구간의 결론

- 성공률은 올랐는데 총량은 내림 — 성공률은 성능의 전부가 아님
- 한 스레드의 정지가 전체의 정지가 되는가(락) vs 그 스레드만의 정지인가(락프리)
- 컨텍스트 스위칭이 오히려 CAS 끼어들기를 줄이고, SMT 짝의 L1 공유가 스냅샷→CAS를 단축
- SRWLock = block 양보 + backoff로 방어

Lock-Free 정리

여기서 다룬 동시성 이해가 이후 네트워크·게임 서버의 멀티스레드 기반

재현·해결한 문제

ABA · Page Decommit
Order Reversal · 공유 풀 오염

얻은 관점

모든 상황에서 락프리가
만능인 것은 아니다

여기서 만든 락프리 스택·큐·메모리 풀은 이후 IOCP 네트워크 라이브러리의 세션 인덱스 관리·송신 큐·메모리 풀에 그대로 사용됩니다.



A* 개요

A* 구현보다 측정 조건 통제와 단계별 최적화 과정이 핵심

측정 공정성

랜덤 맵에서 A*를 반복 측정
BFS로 경로 존재를 먼저 보장

단계별 최적화

휴리스틱 · 캐시 지역성 · 분기 제거
각 단계를 측정으로 판단

측정 표본

5만 회 반복
상·하위 제외 트림 평균

A* 길찾기를 구현했지만, 목표는 성능을 **측정**으로 다루는 것입니다. 경로 존재를 먼저 보장해 측정을 공정하게 만들고, 그 위에서 단계별로 최적화했습니다.

이 프로젝트가 보여주는 것

- 측정 공정성 확보(BFS 선판정)
- 휴리스틱·캐시·분기 3단계 개선
- 5만 회 반복으로 평균 안정화

테스트 설계

랜덤 맵에서 공정하게 측정하기 위한 조건 통제

확률 기반 랜덤 벽 격자 생성



빈 타일에서 시작·도착 무작위 선정



경로 없는 케이스 발생 가능

매 반복마다 **맵(벽 배치)**과 **시작점·도착점**을 전부 새로 랜덤으로 뽑습니다. 맵을 고정하고 지점만 바꾸거나, 맵만 바꾸고 지점을 고정하면 **특정 배치에서만 잘 나오는 결과인지 구분할 수 없기** 때문에, 셋 다 매회 랜덤으로 5만 회를 시도합니다.

확률 기반으로 벽을 배치한 랜덤 격자를 만들고, 벽이 아닌 빈 타일에서 시작점과 도착점을 무작위로 뽑습니다. 랜덤 맵이라 시작→도착 경로가 아예 없는 경우가 생기는데, 이런 케이스가 섞이면 A*가 최악으로 동작해 평균이 왜곡됩니다. 그래서 다음 단계에서 **BFS로 경로 존재를 먼저 검증**하고, 없으면 맵부터 다시 생성합니다.

표본은 **5만 회**로 확보하고, 상·하위 극단값을 제외한 평균으로 안정화했습니다.

BFS 검증 + A* 흐름

경로 존재를 BFS로 먼저 보장한 뒤 A*를 측정 — 측정 구간 분리



□ BFS = 측정 구간 밖 ■ A* = 측정 구간

처리 절차은 맵 생성 → 시작/도착 선정 → BFS 경로 검증 → A* 측정 순으로 흐름니다. BFS는 큐와 방문 배열로 경로 존재만 확인하고, 없으면 맵을 새로 만듭니다.

핵심은 프로파일러가 **BFS 통과 이후에 시작**하도록 해 BFS 시간이 A* 측정에 포함되지 않는다는 점입니다. A*는 우선순위 큐에서 f가 가장 작은 노드를 확장하고, 각 칸에 부모를 저장해 도착 후 경로를 복원합니다.

핵심 포인트

- BFS는 측정 밖 → 순수 A* 비용만 측정
- 경로 있는 케이스만 비교 → 공정성 확보
- 우선순위 큐 + 부모 추적으로 경로 복원

최적화 단계

휴리스틱, 캐시 지역성, 분기 제거 세 단계로 나눈 최적화

유클리드 (sqrt 포함)

제공된 연산 반복 · 대각 이동 미반영



유클리드 휴리스틱

sqrt 제거 · 8방향 대각 이동 반영

흩어진 격자 배치

인접 타일 접근 시 캐시 미스



연속 메모리 배치

구조체 크기 축소 + 2차원 배열 배치 변경

switch 8방향 분기

방향마다 분기 비용



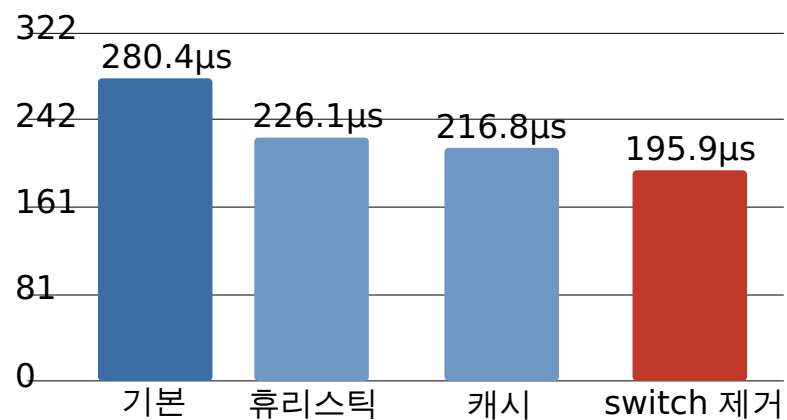
방향·비용 테이블

테이블 참조로 분기 제거

알고리즘 복잡도는 그대로지만, 계산 비용·메모리 배치·분기라는 요인이 실측 성능을 좌우한다는 점을 단계별로 확인했습니다.

성능 개선 결과

측정 기반 3단계로 평균 처리시간 약 30% 감소



5만 회 반복 측정 기준 평균 처리시간은 기본 **280.4μs**에서 시작해 휴리스틱 변경으로 226.1, 캐시 지역성 개선으로 216.8, switch 제거로 **195.9μs**까지 줄었습니다. 최종 약 **30%** 감소입니다.

가장 큰 폭은 휴리스틱 단계에서 나왔는데, 탐색 노드 수 감소와 제공근 제거가 함께 작용했기 때문입니다.

해석 주의: 절대 시간은 PC 사양에 따라 다르므로 같은 환경에서의 상대 개선으로 봐야 합니다.

A* 정리

측정 조건 통제와 단계별 최적화 경험의 서버 성능 분석으로 연결

확보한 것

BFS로 측정 공정성
3단계로 약 30% 감소

얻은 관점

분기문 삭제 시 성능이 상당히
많이 올라가는 것을 실측함

많은 객체·좌표·경로를 반복 처리하는 게임 서버에서 이런 실측 기반 판단은 특히 중요합니다.

IOCP 라이브러리 개요

여러 테스트 서버로 안정성을 검증한 IOCP 네트워크·게임 라이브러리를 직접 구현



특정 게임 로직이 아니라, 여러 서버가 공통으로 쓰는 **IOCP 네트워크·게임 라이브러리**를 직접 구현한 프로젝트입니다. 라이브러리는 IOCP·세션·송수신을 담당하고, 각 서버는 콜백만 구현해 그 위에 올라갑니다.

채팅(단일/멀티/인증)·로그인·모니터링·게임 등 성격이 다른 서버로 검증했으며, **에코 서버는 그중 하나**입니다.

라이브러리 구조

네트워크와 콘텐츠를 분리해, 하나의 코어 위에 여러 서버를 올린 구조



모듈은 세 층으로 나뉩니다. **common_files**는 링버퍼·직렬화 버퍼·TLS 메모리 풀·락프리 자료구조 등 공용 기능을 제공 하고, 그 위에 **network_library**(IOCP 네트워크 코어)와 **game_library**(그룹·프레임 로직)가 **서로 대등한(수평) 모듈**로 나란히 올라갑니다. servers는 이를 상속·사용하는 콘텐츠 층입니다.

이렇게 나눈 이유는 **네트워크와 콘텐츠를 분리해 코어를 재사용**하기 위해서입니다. 절충은 콘텐츠가 콜백 계약 안에서만 확장된다는 점입니다.

Redis는 로그인·인증 서버에서 쓰이고, DB(MySQL)는 로그를 저장합니다. 로그인 서버에서 인증 토큰을 플랫폼과 통신하여 검증하는 것은 오래 걸리기 때문에, 이를 DB 접근으로 대체하였습니다.

검증 서버군 · 콘텐츠와 스레드 구조

서버마다 콘텐츠 성격에 맞춘 스레드 구조 설계

서버	콘텐츠 (무엇을 하나)	스레드 구조
채팅 (싱글)	섹터 기반 채팅방 · 메시지 브로드캐스트	Worker는 OnRecv에서 Job만 큐에 적재 → Update 스레드 1개 가 순차 처리 (콘텐츠 락 없음)
채팅 (멀티)	동일 콘텐츠	Worker 스레드들이 락을 잡고 그 자리에서 직접 처리
채팅·로그인 인증	로그인 토큰 검증 후 입장 허용	Worker 처리 + Redis 토큰 조회
로그인	계정 확인 · 인증 토큰 발급	Worker 처리 + Redis
모니터링	각 서버의 TPS·CPU·메모리·재전송 지표 수집·표시	수집 · 집계
게임 (एको)	game_library의 그룹·프레임 구조 검증	Accept · Worker + 그룹 프레임 Update

서버마다 콘텐츠의 성격이 다르고, 그에 맞춰 **스레드 구조도 다르게** 가져갔습니다.

특히 같은 채팅 콘텐츠를 **싱글(Job 큐 직렬화)**과 **멀티(Worker 직접 처리)** 두 방식으로 구현해 성능을 비교했습니다 — 다음 페이지에서 다룹니다.

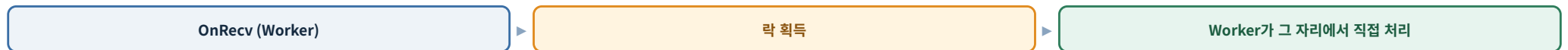
채팅 서버 · 싱글 vs 멀티

같은 콘텐츠를 두 스레드 모델로 비교 — 이 콘텐츠에선 직접 처리가 더 빠름

싱글 — Job 큐 직렬화



멀티 — Worker 직접 처리



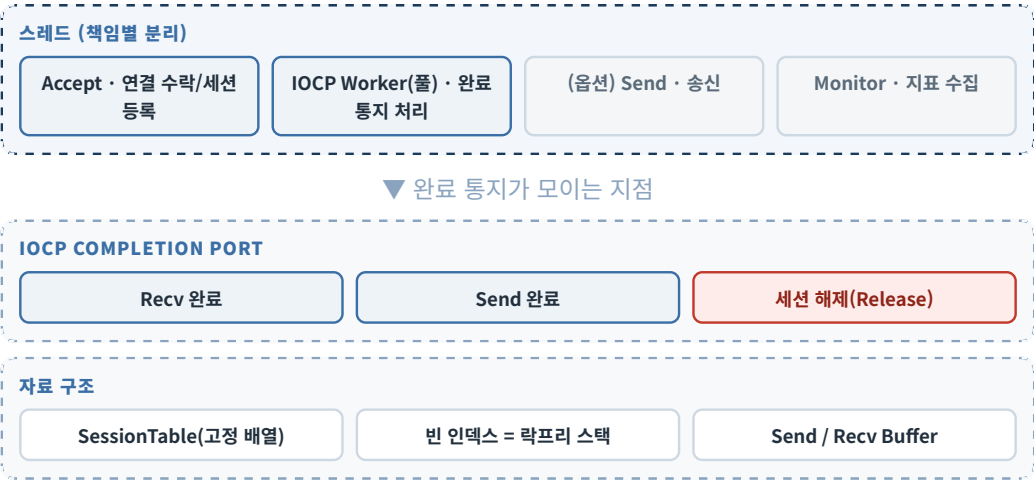
싱글은 Worker가 수신에서 Job만 만들어 큐에 넣고, 콘텐츠 처리는 Update 스레드 하나가 순차로 소화합니다 — 콘텐츠에 락이 없습니다. 멀티는 Worker들이 락을 잡고 받은 자리에서 바로 처리합니다.

같은 부하로 두 서버를 비교한 결과, **멀티 쪽의 Action Delay 평균이 더 낮게** 측정되었습니다. Job 큐를 거치는 처리와 단일 Update 스레드가 메시지를 직렬 처리하면서 지연을 만들고, 락 구간이 짧고 Job 큐에 처리할 메시지가 쌓여 있을 때 이렇게 병렬 처리를 통해 성능을 높일 수 있습니다.

다만 콘텐츠가 복잡해져 락 구간이 길어지면 직렬화 방식의 단순함(락 경험 없음)이 유리해질 수도 있습니다.
근거: 두 서버 동시 부하테스트의 모니터링 지표 캡처.

Thread Architecture

스레드는 책임별로 분리, 세션은 락 없이 안전하게 해제



스레드를 나눈 이유는 각 스레드가 **하나의 책임만** 갖게 하기 위해서입니다. Accept는 연결 수락·세션 등록만, Worker 풀은 완료 통지 처리만 담당합니다.

Worker는 완료 포트에서 이벤트를 꺼내 **Recv / Send / 세션 해제**로 분기합니다. 세션은 고정 배열로 두고 빈 인덱스를 락프리 스택으로 관리하며, 세션 ID에 **배열 인덱스 + 할당 ID**를 함께 인코딩해 조회를 빠르게 합니다. 완료 키에 세션 포인터를 직접 넣어 조회 비용도 없었습니다.

세션 해제는 **참조 카운트 + 릴리즈 플래그**를 한 번의 **128비트 CAS**로 판별해 중복 해제를 막습니다. 이 구조는 뒤의 recv 병목 트러블슈팅과 직접 연결됩니다.

Session Lifecycle

세션의 생성부터 해제까지 락 없이 안전하게 관리



세션 하나의 수명은 수락 → 할당 → 등록 → 수신 등록 → 완료 처리 → 송신 → 종료 → 해제로 흐릅니다.

완료 키에 세션 포인터를 직접 실어 조회 비용을 없앴고, 해제 시점은 참조 카운트와 릴리즈 플래그를 128비트 CAS로 한 번에 판별해 세션을 누가 참조하고 있는지와 누가 Release 했는지를 한 번에 체크하여 Release를 처리합니다.

만약 참조 카운트가 0인지와 Release 체크를 분리하면 문제가 발생합니다.

핵심 포인트

- 세션 ID = 인덱스 + 할당 ID
- 빈 인덱스는 락프리 스택으로 재사용
- 해제는 원자 연산 한 번으로 판별

Recv / Send 흐름

수신은 링버퍼로 조립, 송신은 세션당 WSASend 1회만 유지

Recv



Send



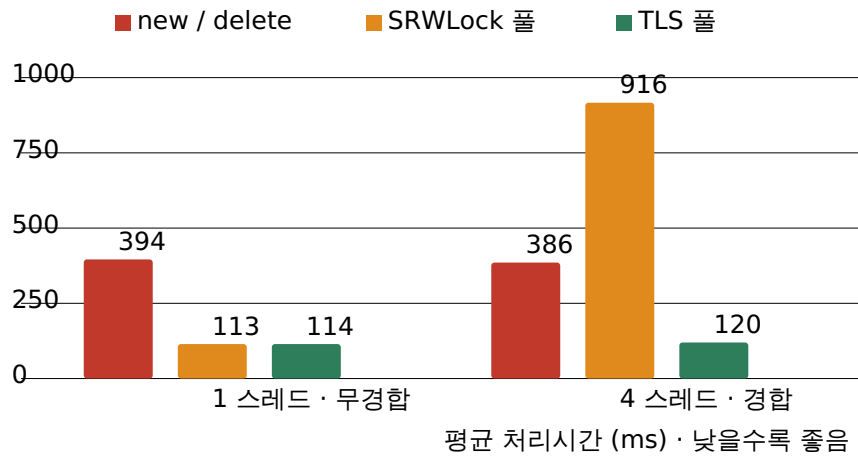
수신은 완료 통지 → 링버퍼 적재 → 헤더 기준 패킷 조립 → **OnRecv 콜백**으로 콘텐츠에 전달 → 다음 수신 등록 순입니다.
외부용 서버는 이때 코드·길이·체크섬을 검증하고 어긋나면 연결을 끊습니다.

송신은 메시지를 세션의 락프리 큐에 넣고, **세션당 WSASend를 1회로 제한**하기 위해 플래그로 제어합니다. 완료되면 반납하고 남은 큐를 이어 보냅니다.

세션당 단일 송신은 **WSASend를 1회로 제한**하여 커널 모드 진입 빈도를 제한하기 위함입니다.

TLS 메모리 풀 성능 측정

세 가지 할당·해제 방식 성능 비교, 경합 구간에서 TLS 풀만 성능 유지



측정으로 확인한 것

- 무경합에서도 new/delete가 **3.4배** 느림. 원자 연산은 세 방식이 모두 사용하므로, 이 차이는 **범용 할당자가 추가로 거치는 경로**에서 발생한 것으로 판단
- 경합 시 new/delete가 SRWLock 풀보다 빠름 → 일반 경로에서 **전역 락으로 직렬화하지 않음**
- 그럼에도 TLS 풀보다 느림 → 호출마다 일정한 **동기화 비용을 계속 지불**

확인하지 못한 것: 힘이 그 비용을 낮추는 내부 방식은 구현 영역이라 검증하지 못했습니다.

방식	1 스레드	4 스레드	4 스레드 변화
new / delete	393.7 ms	386.0 ms	거의 없음
SRWLock 풀	113.1 ms	916.1 ms	8.1배 느려짐
TLS 풀	114.4 ms	119.8 ms	거의 없음

측정 조건: 16바이트 객체를 스레드당 동시 100개 보유, 표본 1회당 10만 반복, 표본 10회 측정. 스레드당 총 1억 회 할당·해제. 물리 코어에 1:1 고정. 할당한 스레드가 직접 해제하는 패턴.

장시간 안정성 테스트

여러 서버를 장시간 무중단으로 돌려 안정성 검증

7일

채팅 서버 무중단

세션 누수·크래시 없이 유지

7일

게임 서버 연결

장시간 연결 안정성

5일

채팅-로그인 인증

인증 경로 지속 검증

부하는 더미 클라이언트로 만들고, 각 서버가 보낸 TPS·CPU·메모리·TCP 재전송 지표를 모니터링 서버에서 관측했습니다.
장시간 테스트의 목적은 순간 성능이 아니라 **오래 유지되는지**입니다. 이 과정에서 원인이 서버 코드 밖(OS)에 있는 문제도 겪었고, 이는 다음 페이지의 트러블슈팅으로 이어집니다.

트러블슈팅 1 · SendQ 폭증 / 완료 통지 중단

IO는 걸려 있는데 완료 통지가 멈춘 문제 — 원인 미확정, OS 재설치로 해소

문제	부하 테스트 중 특정 시점부터 세션들의 SendQ(송신 큐)가 계속 쌓여 폭증.
증상	WSASend는 게시(post)되어 있는데, 송신 완료 통지가 더 이상 오지 않음.
분석	송신 로직 점검 → 세션 Lock 버전으로 테스트 → 하드웨어 순으로 의심하며 추적했으나 코드에서 원인을 찾지 못함. 네트워크 카드 등 하드웨어 교체도 효과 없음.
원인	미확정 — 코드·하드웨어를 배제한 끝에 OS 내부 문제로 추정.
해결	OS를 재설치하자 같은 코드에서 더 이상 재현되지 않음.
결과	이후 장시간 테스트에서 재발 없음.

- 분석의 전환점
- 송신이 완료 통지에 의존하는 구조라 치명적
 - 코드·하드웨어 배제 → OS 내부 추정

라이브러리 → MMORPG 연결

이 네트워크·게임 라이브러리가 MMORPG 월드 서버의 기반

game_library

common_files

MMORPG 월드 서버의 기반

여기서 구현한 것 중 **game_library**와 **common_files**가 MMORPG 월드 서버의 기반 라이브러리로 그대로 사용됩니다(네트워크·세션 처리까지 게임 라이브러리가 담당). 서버의 콘텐츠 그룹은 게임 라이브러리의 그룹 클래스를 상속하고, 유저 객체는 유저 인터페이스를 상속합니다.

한 가지 구분할 점은, 여기서 만든 **게임(에코) 서버**는 **게임 라이브러리를 검증하기 위한 서버**입니다.

핵심 포인트

- game_library·common_files를 그대로 재사용
- 콘텐츠 그룹·유저 객체를 상속
- 에코 서버는 검증용

MMORPG 월드 서버 개요

앞서 만든 라이브러리를 실제 MMORPG 월드 서버에 적용한 대표 결과물



C++ IOCP 기반 MMORPG 월드 서버로, UE5 클라이언트와 연동됩니다. 서버는 AOI·이동 동기화·서버 권위 전투·몬스터 AI·아이템·비동기 DB 저장을 구현했습니다.

핵심은 이 서버가 처음부터 새로 만든 것이 아니라, 앞서 구현·검증한 네트워크·게임 라이브러리를 기반으로 포함하고 그 위에 게임 기능을 올렸다는 점입니다. 단순 기능 구현에 그치지 않고 **최대 4000명** 규모의 부하로 병목을 측정·개선한 과정까지 포함합니다.

라이브러리 기반 적용 구조

라이브러리를 포함하고 그룹을 상속해 게임 콘텐츠로 확장

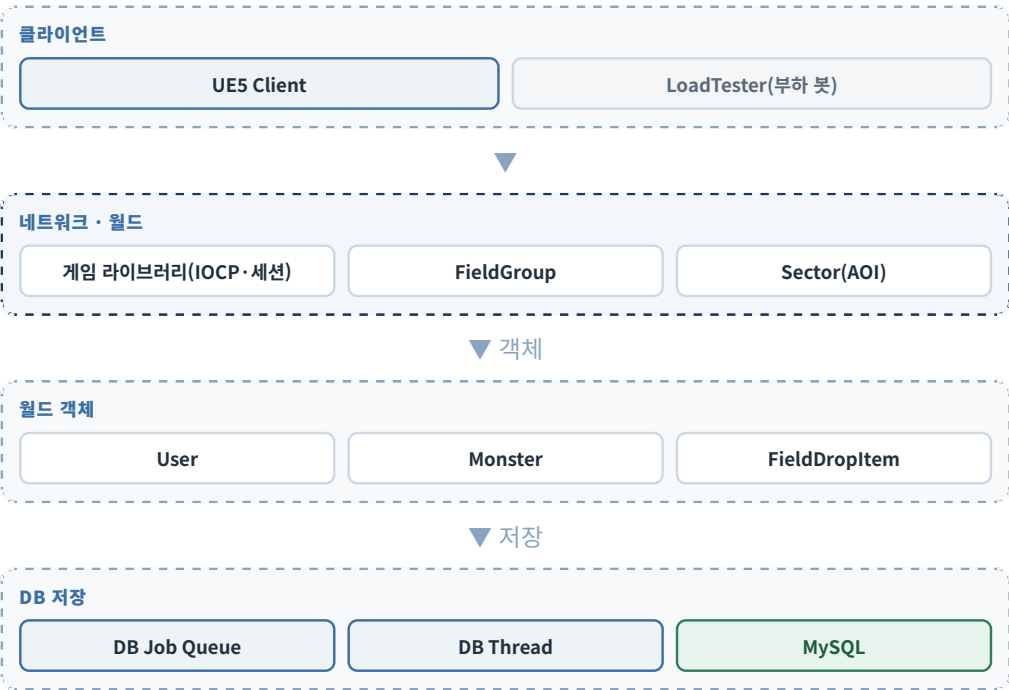


서버 안에는 앞서 만든 기본 라이브러리 중 **game_library**와 **common_files**가 포함되어 있고, 이것이 서버의 기반입니다. **network_library**는 이 서버에서는 쓰지 않고, 네트워크·세션 처리까지 게임 라이브러리가 담당합니다. 콘텐츠 그룹인 **FieldGroup(월드)**과 **AuthGroup(입장)**은 게임 라이브러리의 그룹 클래스를 상속하고, 유저 객체 **CUser**는 유저 인터페이스를 상속합니다.

검증된 네트워크·프레임 기반을 재사용해 콘텐츠에 집중. 라이브러리의 콜백 계약과 그룹·프레임 모델을 그대로 계승.

프로젝트 아키텍처

네트워크·월드·객체·DB 저장을 계층으로 나눈 구성



전체 서버는 클라이언트/부하 도구, 네트워크·월드, 월드 객체, DB 저장의 층으로 구성됩니다. **게임 라이브러리**가 IOCP 네트워크·세션을, **FieldGroup**이 월드 로직을, DB 계층이 저장을 담당합니다.
부하 테스트는 별도 **LoadTester**가 다수 봇으로 접속·행동을 만들고 응답을 검증합니다.

Thread Architecture

메시지·프레임 로직은 그룹 Lock 안의 콜백으로 처리, DB 저장은 전용 스레드로 분리



메시지 처리: Worker가 세션 링버퍼에서 메시지를 꺼내면, 세션에 기록된 그룹의 Lock을 걸고 OnRecv 콜백으로 넘긴 뒤 Lock을 풀니다. 그룹마다 메시지 큐를 두는 것이 아니라, 그룹 Lock으로 직렬 처리하는 구조입니다.

프레임 로직도 그룹의 콜백으로 구현됩니다. Frame 스레드는 그룹 프레임(25ms·40fps)마다 PQCS로 완료 포트에 던지고, IOCP Worker가 해당 그룹의 Lock을 걸고 OnUpdate 콜백을 호출합니다.

DB 스레드를 분리한 이유는, 콘텐츠에서 DB에 바로 접근하면 그동안 콘텐츠 로직 처리가 멈추기 때문입니다. 게임 로직은 DB Job Queue에 enqueue만 하고, 전용 DB 스레드가 꺼내 MySQL에 씁니다. Monitor는 TPS·큐 깊이 등 지표를 주기 수집합니다.

같은 그룹은 Lock으로 직렬 처리, 다른 그룹끼리는 병렬. 이 DB Job Queue의 깊이(dbQueue)가 부하 증가 시 병목 지표가 되며, 뒤의 DB Queue 병목 분석 페이지에서 다룹니다.

클래스 책임 분리

책임을 나누고 데이터 소유권을 한 곳에 두어 정합성 유지

클래스 / 모듈	책임
게임 라이브러리(CGameLibrary)	세션 관리, 패킷 라우팅
GameServer	게임 라이브러리 관리, 그룹 관리, DB 저장 관리
FieldGroup	월드 업데이트, AOI, 전투/아이템 처리
CUser	유저 상태, 이동, 전투, 인벤토리
Monster	AI 상태, 이동, 전투 대상
Sector	AOI 후보 관리
UserItemStorage	실제 아이템 UID 관리
DBManager	DB Job 처리, MySQL 저장

책임을 분리한 이유는 **패킷 처리와 콘텐츠 처리, 데이터 소유를 명확히** 하기 위해서입니다. 네트워크 경계(세션·라우팅)는 **게임 라이브러리**가, GameServer는 라이브러리·그룹·DB 저장의 관리를, FieldGroup은 월드 로직을, 각 객체는 자기 상태를 책임집니다.

특히 아이템 실체는 **UserItemStorage가 UID로 단일** 관리하고 슬롯은 참조만 하게 해, 중복 소유로 인한 정합성 문제를 없앴습니다.

9-Sector AOI

주변 객체 후보를 섹터 단위로 제한하고, 이동 시 차집합만 갱신



대각(우상) 이동 1섹터 — 이탈 5 · 진입 5 · 유지 4

월드를 섹터 격자로 나누고 유저·몬스터·드랍 아이템을 섹터 단위로 관리합니다. 내 위치를 중심으로 한 **9섹터(3×3)**가 시야입니다.

캐릭터가 섹터를 옮기면 이전 시야와 새 시야의 **차집합만** 계산합니다. 그림처럼 대각으로 1섹터 이동하면 **이탈한 5개 섹터에 Delete**, **새로 보이는 5개 섹터에 Create**를 보내고, 겹치는 4개 섹터는 그대로 둡니다.

전체 브로드캐스트 대비 트래픽이 크게 줄고 계산이 단순. 섹터 크기가 시야 반경과 결합돼 조절이 까다롭고 경계 처리 주의가 필요합니다(→ 몬스터 생성/삭제 중복 트러블슈팅과 연결).

이동 동기화

좌표를 임계 범위 안에서는 신뢰, 벗어나면 서버 좌표로 강제 보정

서버 좌표와 클라 좌표의 차이 검사

임계 내 · 신뢰

클라 좌표 신뢰 → 섹터 갱신 → 브로드캐스트

임계 밖 · 강제 싱크

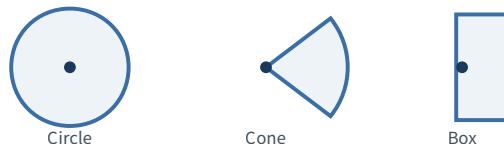
서버 좌표로 강제 보정(싱크) → 브로드캐스트

이동은 클라이언트가 **좌표·방향·이동 플래그**를 보내며 시작합니다. 서버는 자신이 아는 좌표와의 차이를 검사해, 임계 범위 안이면 클라 좌표를 신뢰하고 벗어나면 서버 좌표로 강제 보정합니다.

서버가 클라 이동 요청을 받아 응답을 주어 클라가 이동하는 구조는 클라 조작감이 좋지 않아, **이동 패킷은 클라가 먼저 보내는 방식**으로 만들었습니다.

서버 권위 전투

클라이언트는 방향만 전송, 피격 대상과 데미지는 서버가 판정



클라이언트는 공격 방향과 스윙 종류(또는 스킬 슬롯)만 보냅니다. 서버가 사거리에 걸치는 섹터를 추려 그 안의 대상만 검사하고, 타격 범위를 원·부채꼴·직사각형으로 판정합니다.

거리·각도 비교는 제곱을 이용해 제곱근 연산을 피합니다. 이어서 데미지·사망·경험치·드랍을 서버가 계산하고 결과를 주변에 브로드캐스트합니다.

프로토콜 설계

클라이언트는 입력만, 확정은 서버가 — UID와 타인 HP는 숨김

프로토콜	클라가 보내는 값	서버가 판단	숨긴 값 / 이유
이동	좌표·방향·플래그	싱크 임계 검증·보정	— · 클라가 먼저 이동(응답성)
전투	공격 방향·스윙	타겟·데미지	타겟 리스트 · 치팅 방지
스킬	슬롯	MP·쿨다운·사거리	— · 서버 권위
아이템	슬롯 타입·인덱스	UID 처리·결과	아이템 UID · 외부 노출 방지
필드 드랍	—	DropID→줍는 순간 ItemUID 부여	— · 임시/영구 생명주기 분리
캐릭터 초기화	—	인벤토리·장비 목록 전송	타 캐릭터 정확 HP(피격 시 비율만)

원칙은 하나입니다. 클라이언트는 입력만, 확정은 서버가 합니다. 다만 이동은 서버 응답을 기다렸다 움직이는 게 아니라 클라 이언트가 먼저 움직이고 서버가 사후 검증·보정합니다 — 응답성 때문입니다. 아이템 UID는 외부에 노출하고 싶지 않아 서버 내부에서만 사용합니다.

타 캐릭터의 HP/MaxHP도 장비 장착 및 해제, 레벨 등 이벤트 발생 시마다 주변에 전달해서 동기화하는 것이 아니라, 피격 시 비율만 전송해 정보 노출과 트래픽을 줄입니다.

몬스터 AI

상태 머신과 시간 조건으로 안정적인 몬스터 동작 구현



전이	조건
Idle/Patrol → Chase	플레이어 감지
Chase → Combat	공격 범위 진입
Combat → Chase	공격 범위 이탈 상태가 일정 시간 지속(시간 조건)
→ Return	타겟 상실
→ Dead	사망
Return → Idle	복귀 완료

몬스터는 6상태 머신으로 동작하며 상태별 진입·깽신을 분리했습니다. 특히 추격과 전투 사이는 매 프레임 오가는 떨림이 생기기 쉽습니다.

그래서 전투 이탈을 범위를 벗어나는 즉시가 아니라, 벗어난 상태가 일정 시간 지속될 때만 추격으로 전환하도록 시간 조건을 뒀습니다.

아이템 / Storage

아이템 실체는 한 곳에서 UID로 관리, 슬롯은 참조만 — 정합성 유지



아이템의 실체는 **UserItemStorage가 UID로 단일 관리**합니다. Inventory·Equipment·QuickSlot은 슬롯에 UID만 담는 뷰이고, 실제 데이터는 Storage에만 있습니다. 데이터 소유권을 한 곳에 두면 중복 소유가 없어 정합성이 유지됩니다.

필드 드랍은 **DropID**로만 관리하다가, 유저가 주워 Storage에 들어가는 순간 **아이템 UID**를 부여합니다. 돌을 나눈 이유는 필드 드랍(임시)과 유저 소유(영구)의 생명주기가 다르기 때문입니다.

DropID → ItemUID 생명주기



아이템 DB 테이블 설계

아이템 테이블을 겹침·장비 개체·랜덤 스탯 셋으로 분리 — 불필요한 컬럼·갱신 이상 제거

Before · 통짜 아이템 테이블

item (통짜 테이블)

itemUID · characterUID · itemID
slottype · slotindex · count
ATK · DEF ... (장비 전용)
randomStat1 · randomStat2 ...

포션 같은 겹침 아이템도 장비 전용 컬럼(ATK·DEF·랜덤 스탯)을 들고
감 — 불필요한 컬럼과 빈 값



After · 3테이블 분리

stackitem

itemUID (PK)
characterUID · itemID
slottype · slotindex
count

instanceitem

itemUID (PK)
characterUID · itemID
slottype · slotindex

instanceitem_stat

itemUID + randomStatType (PK)
statValue

성격별 분리: 겹침(수량) / 장비 개체 / 랜덤 스탯 — 슬롯 이동은
instanceitem 1행만 갱신

처음에는 아이템 테이블이 하나라서, 포션 같은 **겹침(스택) 아이템**을 넣을 때도 장비에만 필요한 ATK·DEF 컬럼을 함께 들고 갔습니다. 그래서 수량이 있는 아이템은 **stackitem**, 장비 개체는 **instanceitem**으로 분리해 불필요한 컬럼을 없앴습니다.

그런데 랜덤 스탯을 instanceitem에 함께 두면 스탯 수만큼 한 아이템에 대해 **여러 행**이 삽입됩니다. 슬롯 위치 하나를 바꾸는 데도 그 아이템의 행을 전부 찾아 바꿔야 하고, 하나라도 놓치면 **갱신 이상**이 납니다. 그래서 스탯을 **instanceitem_stat**(키: itemUID+randomStatType)으로 다시 분리해, 슬롯 이동은 instanceitem 1행 갱신으로 끝나게 했습니다.

로드는 instanceitem과 instanceitem_stat을 **LEFT JOIN 한 번**으로 읽어 개체와 스탯을 함께 복원합니다. 아이템 UID는 uid_sequence 테이블에서 구간 단위로 예약해 발급합니다.

DB Job Queue

게임 루프와 DB 저장을 분리해, 저장 지연이 로직을 막지 않는 구조



DB 저장은 느리기 때문에 게임 루프에서 직접 하면 서버가 막힙니다. 그래서 컨텐츠 로직이 저장할 내용을 **DB Job**으로 만들어 큐에 넣고, 전용 DB 스레드가 꺼내 MySQL에 씁니다.

이때 여러 쓰기 Job을 **한 트랜잭션으로 묶어 커밋(배치)**합니다.

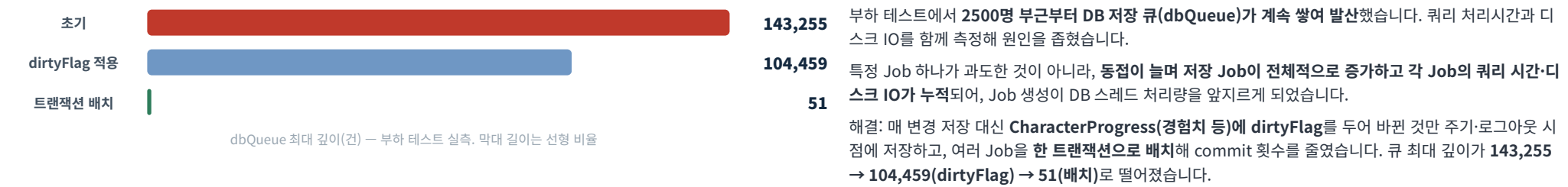
저장이 비동기라 지연이 생기고 크래시 시 미반영분이 유실될 수 있음. 이 큐의 깊이(dbQueue)가 병목 관측 지표가 되며 다음 페이지에서 다룹니다.

핵심 포인트

- 게임 루프는 enqueue만
- DB 스레드가 dequeue → MySQL
- 여러 Job을 한 트랜잭션으로 배치

DB Queue 병목 분석

동접 증가로 저장 요청·쿼리 시간·디스크 IO가 누적되어 큐 폭증



변경분만 저장

CharacterProgress(경험치 등)에 dirtyFlag 바뀐 것만 주기 저장 + 로그아웃 시 저장

트랜잭션 배치

여러 쓰기 Job을 한 트랜잭션으로 커밋
commit·디스크 IO 횟수 감소

결과

dbQueue 최대 143,255 → 51
2500명 발산 해소

트러블슈팅 1 · 몬스터 생성/삭제 중복

몬스터별 이벤트 히스토리로, 사망 Delete가 자기 섹터에만 가던 전파 누락을 발견

문제	클라이언트에서 이미 있는 몬스터가 다시 생성되거나, 없는 몬스터의 삭제가 옴.
증상	특정 상황에서 중복 Create / 모르는 Delete 발생.
분석	‘그 순간 상태’로는 원인을 알 수 없어, 몬스터별로 Create/Delete 이벤트 히스토리(종류+위치)를 메모리 배열에 기록해 시퀀스를 복원.
원인	몬스터 사망 시 Delete를 주변 섹터가 아니라 자기 섹터에만(반복해서) 전송 — 주변 섹터의 유저는 Delete를 받지 못함.
해결	사망 Delete를 주변(AOI) 섹터까지 정확히 전파하도록 수정.
결과	중복 생성·유령 몬스터 해소.
배운 점	락프리 디버깅에서 쓴 이벤트 메모리 기록 방식이 콘텐츠 버그 추적에도 그대로 유효하다.

핵심 포인트

- 핵심 = 몬스터별 이벤트 히스토리 배열(종류+위치)
- 원인 = 사망 Delete가 자기 섹터에만 반복 전송
- 락프리 디버깅 기법의 재사용

트러블슈팅 2 · BadValue (스탯 매핑 실수)

더미의 값 범위 검증이 잡아낸 이상 값 — 원인은 DB Job의 컬럼 매핑 실수

문제	부하 도구의 값 범위 검증에서 몬스터 공격력이 비정상 값으로 검출(BadValue).
증상	설계값은 ATK 1~3 · 2~4인데 13·26·29가 관측되고, DB에는 17이 저장되어 있음.
분석	값이 만들어져 흐르는 경로(생성 → DB 저장 → 로드 → 전파)를 따라 추적.
원인	DB 저장 Job을 만들 때의 컬럼 매핑 실수 — max_hp를 atk에, max_mp를 def에 넣고 있었음.
해결	매핑을 바로잡음.
결과	스탯이 설계값 범위로 정상화.

핵심 포인트

- 부하 도구의 값 범위 검증이 오류 검출
- 생성→저장→로드→전파 흐름을 추적
- 원인 = max_hp→atk · max_mp→def 매핑

트러블슈팅 3 · HP BadValue (미초기화)

HP 이상 값의 원인은 초기화되지 않은 공격·방어 스탯

문제	부하 도구의 값 범위 검증에서 HP가 비정상 값으로 검출(BadValue).
증상	데미지 계산 결과가 어긋나 HP가 정상 범위를 벗어남.
분석	데미지 계산에 들어가는 값들을 추적 → 장비의 공격·방어 스탯이 초기화되지 않은 채 계산에 사용됨을 확인.
원인	공격·방어 스탯 미초기화.
해결	초기화 누락을 보완.
결과	HP·데미지가 정상 범위로 계산.
배운 점	멤버 변수 초기화 기억하기.

핵심 포인트

- 부하 도구 값 검증으로 검출
- 원인 = 공격·방어 스탯 미초기화

트러블슈팅 4 · Chase/Combat 떨림

이탈 지속 시간 조건으로 상태 진동 차단

문제	몬스터가 추격(Chase)과 전투(Combat)를 매 프레임 오가며 떨림.
증상	경계 근처에서 상태가 진동.
분석	전투 진입은 공격 범위 안, 이탈은 같은 범위 밖으로 판정하면 경계 근처에서 반복됨을 확인.
원인	범위를 벗어나는 즉시 전환해 경계에서 진동.
해결	전투 이탈 판정에 지속 시간 조건을 둬 벗어난 상태가 일정 시간 유지될 때만 전환.
결과	경계 근처의 상태 진동이 사라짐.
배운 점	임계 기반 상태 전환은 시간 조건을 뒤야 진동을 막는다.



트러블슈팅 5 · 스냅샷 보간

렌더링 시점을 서버 시간보다 살짝 이전으로 잡아, 받아 둔 스냅샷 사이를 보간

문제	다른 캐릭터의 움직임이 끊기거나 순간적으로 뒤로 튕(역행).
증상	타 캐릭터 렌더링이 부자연스러움.
분석	레이턴시가 있는 이상, 서버 위치를 받는 즉시 그러면 완벽한 동기화가 불가능함을 확인.
원인	도착 간격이 일정하지 않은 이동 메시지를 즉시 반영해 끊김·역행이 발생.
해결	클라이언트가 이동 메시지를 큐에 모아 두고, 렌더링 시점을 서버 시간보다 살짝 이전으로 잡아 큐에 쌓인 스냅샷 사이를 보간(지연 렌더링).
결과	타 캐릭터 움직임이 부드러워짐.
배운 점	클라이언트 위치 동기화 방법을 배웠음 — 지연 렌더링 + 스냅샷 보간.

역할 분담

- 서버 = 위치 스냅샷 + 타임스탬프 제공
- 클라 = 지연 렌더링 + 스냅샷 보간
- 레이턴시 하에서 ‘즉시 반영’은 부자연스러움

트러블슈팅 6 · 3500명 Sync 폭증

메시지를 모아 보내 세그먼트 수를 줄이자, TCP 재전송과 Sync가 함께 감소

문제	동접 3500명부터 Sync(위치 강제 보정) 카운트가 급증.
증상	모니터링에서 SyncCount 폭증과 함께 더미 클라이언트 머신의 TCP 재전송 증가가 관측됨.
분석	지표를 함께 관측 → 메시지를 날개로 send해 세그먼트 수가 폭증 , 고동접에서 TCP 재전송을 유발하고 패킷이 지연되어 좌표 차이가 싱크 임계를 넘음.
원인	과다한 세그먼트 → TCP 재전송 → 전달 지연 → 서버·클라 좌표 차이 확대 → Sync 급증.
해결	Send 전용 스레드 를 두고 세션별로 쌓인 메시지를 모아 한 번에 전송 — 세그먼트 수를 줄임.
결과	TCP 재전송과 Sync가 함께 감소. 4000명에서 남는 일부 Sync는 더미(봇)의 보정 처리 누락에 의한 것으로 확인.
배운 점	고동접에서는 보내는 바이트양만이 아니라 세그먼트 수 자체 가 병목이 된다.

핵심 포인트

- 재전송·Sync 지표를 함께 관측
- 원인 = 세그먼트 수 폭증
- Send 전용 스레드로 병합 전송

핵심 요약

만들고, 측정하고, 병목을 개선한 과정을 네 프로젝트에 정리

약 30%

A* 처리시간 감소

280.4 μ s → 195.9 μ s, 5만 회 측정

4000명

MMORPG 최대 부하

부하로 병목 측정

7·7·5일

무중단 안정성

채팅 7 · 게임 7 · 인증 5일

143,255→51

DB 큐 최대 깊이

dirtyFlag · 주기 저장·트랜잭션 배치

Lock-Free에서 동시성 문제 4종을 재현·해결하며 멀티스레드 기반을 다졌고, A*에서 측정 공정성을 확보한 뒤 평균 처리시간을 약 30% 줄였습니다.

IOCP 네트워크·게임 라이브러리를 구현해 여러 서버로 **채팅 7일·게임 7일·인증 5일** 무중단을 검증했습니다. 그 라이브러리를 실제 MMORPG 월드 서버에 적용하고 **최대 4000명** 부하로 병목을 측정해, 2500명 부근의 DB 큐 폭증을 dirtyFlag·주기 저장·트랜잭션 배치로 **최대 깊이 143,255 → 51**까지 줄였고, 3500명부터의 Sync 폭증은 **Send 전용 스레드의 메시지 병합(세그먼트 감소)**으로 해결했습니다.

시연 영상 링크

문제의 재현과 해결, 게임 플레이를 영상으로 확인

프로젝트 / 문제	재현 영상	해결 영상
Lock-Free · ABA	youtu.be/tbD-PNxHf5s	youtu.be/LjmOqV8cS9M
Lock-Free · Order Reversal	youtu.be/gUviydSV0wA	youtu.be/7gOG7qrjBoM youtu.be/_QrC7sH-VYo youtu.be/uBlvLsggcAw
Lock-Free · Page Decommit	youtu.be/tFWJiXKYj5Y	youtu.be/pd9q8vR92S4
Lock-Free · Shared Node Pool	youtu.be/iPoZmHwS0OM youtu.be/lyE1E4G3WSU	youtu.be/Jkn-RgcsvVc youtu.be/5qYe9mjRsSk
A* · 길찾기 자동/수동 시물	youtu.be/ui0-Qzgyubk	—
MMORPG · 솔로 플레이 시연	youtu.be/T0bpjV_1AWo	—
MMORPG · 더미 접속 후 플레이	youtu.be/d3HYkCiPdms	—

Lock-Free 영상은 각 문제를 재현 장치로 실제 크래시·오동작까지 재현한 뒤, 해결 코드에서 같은 시나리오가 통과함을 보여줍니다. PDF 뷰어에서 링크를 바로 열 수 있습니다.